

OBJECTIVE: To be familiar with Panda's library and Matplotlib

THEORY:

1. Introduction to pandas

Pandas is an open-source Python library used for data manipulation and analysis. It provides powerful, flexible data structures that make handling structured data easy and efficient.

Key Features

- Fast and efficient data handling
- Easy handling of missing data
- Data alignment and reshaping
- Integration with other libraries like NumPy
- Supports reading/writing data from multiple formats (CSV, Excel, SQL, JSON)

2. Introduction to Matplotlib

Matplotlib is a widely used Python library for data visualization. It helps in creating static, animated, and interactive graphs.

Key Features

- Simple and flexible plotting
- Supports multiple plot types
- Customizable graphs (labels, colours, styles)
- Works well with NumPy and Pandas

Q1. A tourism agency in Nepal maintains its booking records in a CSV file named `tourism_bookings.csv`. The file contains the following columns: `Tour_Package`, `District`, `Bookings`, and `Earnings`. Write a Python program using Pandas to:

- Load the data from the CSV file into a DataFrame.**
- Display the first five rows of the DataFrame.**
- Display basic information about the DataFrame (like shape, column types, memory usage).**
- Replace any missing values in the Earnings column with the median value of that column.**
- Calculate and display the total earnings for each Tour_Package.**
- Filter and display only the records from the Kaski district where the number of Bookings is greater than 15.**

→ code:

```
import pandas as pd

df = pd.read_csv("tourism_bookings.csv")

print("First five rows:")

print(df.head())

print("\nBasic Information:")
```

```

print("Shape:", df.shape)
print("\nData Types:")
print(df.dtypes)
print("\nDetailed Info:")
print(df.info())

median_earnings = df['Earnings'].median()
df['Earnings'].fillna(median_earnings, inplace=True)

total_earnings = df.groupby('Tour_Package')['Earnings'].sum()
print("\nTotal Earnings for each Tour Package:")
print(total_earnings)

filtered_data = df[(df['District'] == 'Kaski') & (df['Bookings'] > 15)]
print("\nFiltered Data (Kaski & Bookings > 15):")
print(filtered_data)

```

Output:

First five rows:

	Tour_Package	District	Bookings	Earnings
0	Annapurna Trek	Kaski	20	50000.0
1	City Tour	Kathmandu	10	20000.0
2	Jungle Safari	Chitwan	25	60000.0
3	Annapurna Trek	Kaski	18	NaN
4	City Tour	Kaski	30	40000.0

Basic Information:

Shape: (6, 4)

Data Types:

Tour_Package object

District object

Bookings int64

Earnings float64

dtype: object

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 6 entries, 0 to 5

Data columns (total 4 columns):

```
# Column      Non-Null Count  Dtype
---  -
0  Tour_Package  6 non-null    object
1  District      6 non-null    object
2  Bookings      6 non-null    int64
3  Earnings      5 non-null    float64
dtypes: float64(1), int64(1), object(2)
memory usage: 320.0+ bytes
None
```

Total Earnings for each Tour Package:

Tour_Package

Annapurna Trek 90000.0

City Tour 60000.0

Jungle Safari 90000.0

Name: Earnings, dtype: float64

Filtered Data (Kaski & Bookings > 15):

```
   Tour_Package District Bookings Earnings
0  Annapurna Trek  Kaski      20  50000.0
3  Annapurna Trek  Kaski      18  40000.0
4    City Tour    Kaski      30  40000.0
```

Q2. Two small DataFrames are given:

import pandas as pd

```
students = pd.DataFrame({
    'roll_no': ['S01', 'S02', 'S03', 'S04', 'S05'],
    'name': ['Aarav', 'Bipana', 'Chirag', 'Dipesh', 'Esha'],
    'class': [10, 10, 9, 10, 9]})
```

```
marks = pd.DataFrame({
    'roll_no': ['S01', 'S03', 'S04', 'S06'],
    'maths': [85, 92, 78, 88],
    'science': [90, 87, 82, 95]})
```

Combine the two DataFrames using pd.merge() with how='left' so that all students appear, even if they have no marks recorded yet. Print the result and comment on what happens to students S02 and S05.

- Now use `pd.concat()` to append the marks DataFrame below the students DataFrame (vertical concat). Print it and explain in a comment why this usually doesn't make sense for this kind of data.
- Use `pd.concat()` again, but this time horizontally (`axis=1`). Print the result and comment on the alignment of rows (which students get matched correctly and why some values become NaN).
- Summarize in your own words (2–3 lines): When should you use `merge()` instead of `concat()`?

→ code:

```
import pandas as pd
```

```
students = pd.DataFrame({
    'roll_no': ['S01', 'S02', 'S03', 'S04', 'S05'],
    'name': ['Aarav', 'Bipana', 'Chirag', 'Dipesh', 'Esha'],
    'class': [10, 10, 9, 10, 9]})
```

```
marks = pd.DataFrame({
    'roll_no': ['S01', 'S03', 'S04', 'S06'],
    'maths': [85, 92, 78, 88],
    'science': [90, 87, 82, 95]})
```

```
merged = pd.merge(students, marks, on='roll_no', how='left')
```

```
print("Merge operation:")
```

```
print(merged)
```

```
vertical_concat = pd.concat([students, marks])
```

```
print("\nvertical concat:")
```

```
print(vertical_concat)
```

```
horizontal_concat = pd.concat([students, marks], axis=1)
```

```
print("\nhorizontal concat:")
```

```
print(horizontal_concat)
```

Output:

Merge operation:

	roll_no	name	class	maths	science
0	S01	Aarav	10	85.0	90.0
1	S02	Bipana	10	NaN	NaN
2	S03	Chirag	9	92.0	87.0
3	S04	Dipesh	10	78.0	82.0

4 S05 Esha 9 NaN NaN

vertical concat:

	roll_no	name	class	maths	science
0	S01	Aarav	10.0	NaN	NaN
1	S02	Bipana	10.0	NaN	NaN
2	S03	Chirag	9.0	NaN	NaN
3	S04	Dipesh	10.0	NaN	NaN
4	S05	Esha	9.0	NaN	NaN
0	S01	NaN	NaN	85.0	90.0
1	S03	NaN	NaN	92.0	87.0
2	S04	NaN	NaN	78.0	82.0
3	S06	NaN	NaN	88.0	95.0

horizontal concat:

	roll_no	name	class	roll_no	maths	science
0	S01	Aarav	10	S01	85.0	90.0
1	S02	Bipana	10	S03	92.0	87.0
2	S03	Chirag	9	S04	78.0	82.0
3	S04	Dipesh	10	S06	88.0	95.0
4	S05	Esha	9	NaN	NaN	NaN

Comment:

i) how='left' → keeps all rows from students

Students S02 and S05:

- No matching roll_no in marks
- So, maths and science become NaN

ii) Rows are simply stacked

- Columns don't match → many NaN values

Why this doesn't make sense?

- students and marks represent different attributes of same entities
- Vertical concat treats them as separate rows, not related data

iii) Alignment is based on index (0,1, 2,...), NOT roll_no. So,

- Row 1 → S02 gets marks of S03 (wrong match)
- Row 3 → S04 gets marks of S06

- Last row has NaN because marks have fewer rows

This causes incorrect data association

iv) Final summary:

- Use merge() when combining data based on a common key (like roll_no)
- Use concat() when stacking or joining datasets without key-based relationship
- merge() ensures correct alignment, while concat() may lead to misaligned or meaningless data

Q3. Write a Python program using Matplotlib to create the following plots

1. Line Plot
2. Bar Chart
3. Scatter Plot
4. Histogram

Consider the data as: $x = [1, 2, 3, 4, 5]$ $y = [10, 20, 15, 25, 30]$. Include proper titles and axes.

→ code:

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 20, 15, 25, 30]
```

1. Line Plot

```
plt.figure(figsize=(6,4))
```

```
plt.plot(x, y, marker='o', color='b', linestyle='-')
```

```
plt.title("Line Plot of x vs y")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.grid(True)
```

```
plt.show()
```

2. Bar Chart

```
plt.figure(figsize=(6,4))
```

```
plt.bar(x, y, color='orange')
```

```
plt.title("Bar Chart of x vs y")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

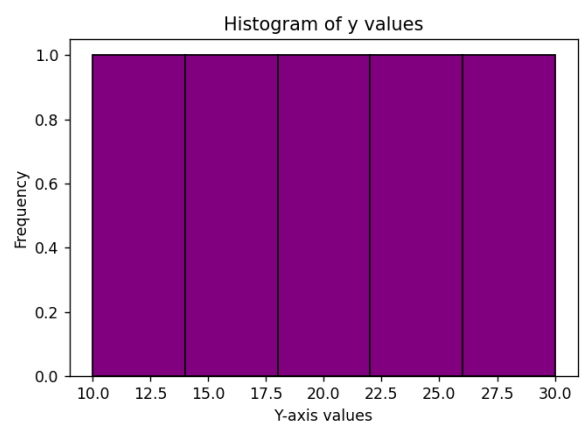
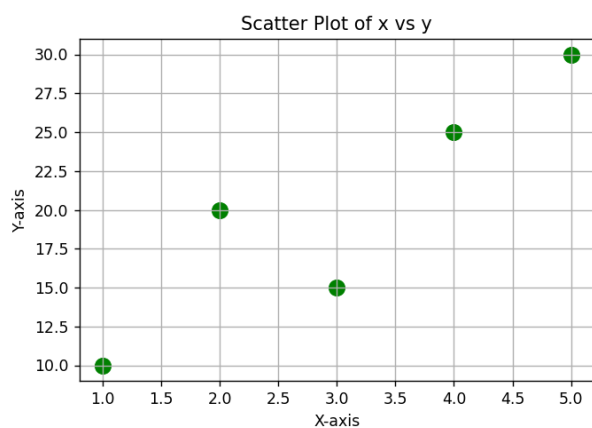
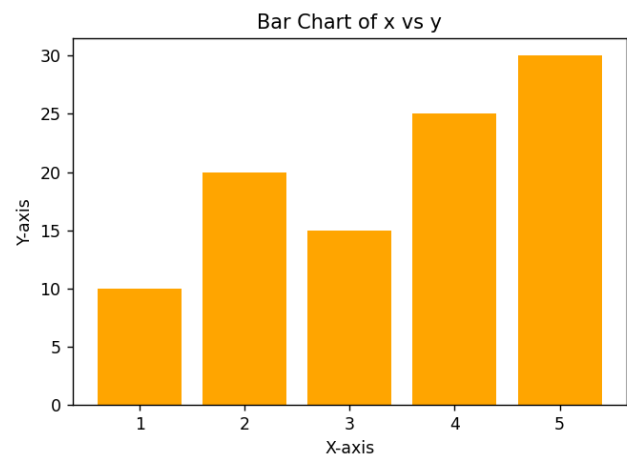
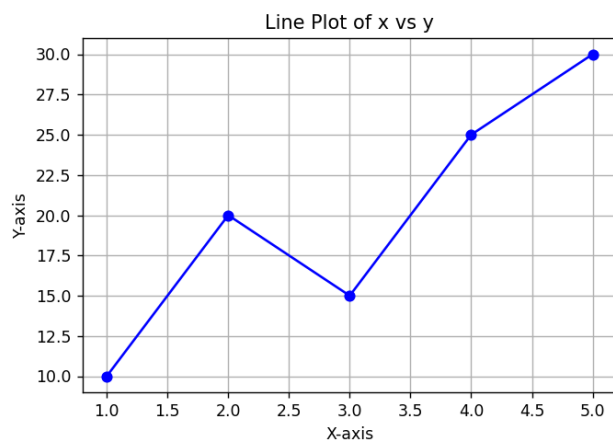
```
plt.show()
```

3. Scatter Plot

```
plt.figure(figsize=(6,4))
plt.scatter(x, y, color='green', s=100)
plt.title("Scatter Plot of x vs y")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()
```

4. Histogram

```
plt.figure(figsize=(6,4))
plt.hist(y, bins=5, color='purple', edgecolor='black')
plt.title("Histogram of y values")
plt.xlabel("Y-axis values")
plt.ylabel("Frequency")
plt.show()
```



RESULT:

The experiment was successfully performed using Pandas and Matplotlib libraries in Python.

- The CSV file containing tourism booking data was successfully loaded into a DataFrame, and its structure, data types, and summary information were displayed correctly. Missing values in the *Earnings* column were handled by replacing them with the median value, ensuring data consistency.
- The total earnings for each tour package were calculated accurately using groupby operations. Filtering operations were also successfully applied to extract specific records (Kaski district with bookings greater than 15).
- DataFrame operations such as `merge()` and `concat()` were performed and analyzed. It was observed that:
 - `merge()` correctly combined related datasets using a common key.
 - `concat()` (vertical and horizontal) resulted in misaligned or incomplete data when used improperly.
- Various plots including line plot, bar chart, scatter plot, and histogram were successfully generated using Matplotlib. These visualizations clearly represented the given data and demonstrated different ways of presenting information graphically.

CONCLUSION:

This lab experiment helped in understanding the practical use of Pandas for data manipulation and Matplotlib for data visualization. It demonstrated how to handle real-world data tasks such as cleaning missing values, grouping, filtering, and combining datasets effectively.

The experiment also highlighted the importance of choosing the correct method (`merge` vs `concat`) when working with multiple datasets to avoid incorrect data alignment. Additionally, the creation of different types of plots improved the understanding of how data can be visually interpreted.

Overall, the lab enhanced practical knowledge of data analysis and visualization in Python, which is essential for fields like data science, machine learning, and analytics.